ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА

ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №3

«Ітератори, генератори та потокова обробка даних»

з курсу «Професійний Python»
Варіант №1 — «Потокова обробка даних системи обліку студентів»

Виконала: Руссу Анастасія Сергіївна

Група: ФЕП-32

Перевірив: Куньо І. М.

Дата: 26.10.2026

Львів — 2026

1. Мета роботи

Навчитися працювати з ітераторами та генераторами Python, використовувати yield, yield from і засоби itertools. Побудувати потоковий конвеєр обробки CSV, виконати фільтрацію, пошук, перетворення, агрегацію та обробку пакетами. Порівняти eager і lazy за часом, піковою пам'яттю та часом до першого результату.

2. Індивідуальний варіант і постановка задачі

Варіант №1 — «Потокова обробка даних системи обліку студентів». Необхідно читати й перевіряти записи CSV, відбирати студентів за групою та мінімальним середнім балом, шукати за ID або прізвищем. Обчислити середній, мінімальний і максимальний бал без повного завантаження даних, визначити найкращих N студентів за умовою, обробити записи пакетами та підрахувати студентів у групах.

2.1. Вхідні дані

Формат CSV містить поля student_id, first_name, last_name, group, average_grade. Малий файл sample.csv складається з 10 записів, серед яких 6 коректних і 4 некоректні. Основний файл students.csv містить 100 000 згенерованих записів. Для експерименту окремо створюються тимчасові файли на 10 000, 100 000, 500 000 і 1 000 000 записів.

У великих наборах ID послідовні й унікальні, використано чотири групи та бали від 40 до 100. Генератор Random(1) забезпечує відтворюваність даних. У довільному CSV перевіряється додатний ID, наявність імені, прізвища й групи, скінченність бала та діапазон 0–100. Глобальна перевірка повторів ID не виконується, оскільки потребувала б зберігання всіх ключів.

```
student_id,first_name,last_name,group,average_grade
1, марія , дорош ,ФЕП-32,86.5
2,Данило,Левицький,ФЕП-31,95
3,Соломія,Климчук,ФЕП-32,91.5
4,Остап,Мазур,ФЕП-31,78
5,Олена,Романюк,ФЕП-33,85
6,Тарас,Дорош,ФЕП-33,72
7,Іван,Тест,ФЕП-32,error
8,Іван,Тест,ФЕП-32,105
9,,Тест,ФЕП-32,80
10,Іван,Тест,ФЕП-32,nan
```

3. Структура проєкту

```
Lab_03_variant_1/
├── pyproject.toml
├── README.md
├── .gitignore
├── data/
│   ├── sample.csv
│   └── students.csv
├── src/
│   └── stream_processor/
│       ├── __init__.py
│       ├── readers.py
│       ├── parsers.py
│       ├── filters.py
│       ├── iterators.py
│       ├── pipeline.py
│       ├── analytics.py
│       ├── data.py
│       ├── main.py
```

```
│       └── benchmark.py
└── tests/
    └── test_pipeline.py
```

Модулі readers, parsers і filters утворюють послідовні етапи конвеєра. pipeline поєднує їх, analytics виконує обчислення, iterators формує пакети. Модуль data генерує CSV, benchmark проводить експеримент, main керує запуском і виведенням.

4. Опис створених модулів

Iterable — об'єкт, із якого iter() отримує ітератор. Iterator зберігає стан обходу, повертає наступне значення через next() і завершує обхід винятком StopIteration. Для ітератора iter(iterator) повертає сам об'єкт. Повторне використання вичерпаного ітератора не починає обхід заново.

4.1. Власний ітератор пакетів

```
from collections.abc import Iterable, Iterator
from itertools import count, islice
from stream_processor.parsers import Student
class BatchIterator:
    def __init__(self, students: Iterable[Student], size: int):
        if size <= 0:
            raise ValueError("Розмір пакета має бути додатним.")
        self.iterator = iter(students)
        self.size = size
    def __iter__(self):
        return self
    def __next__(self) -> list[Student]:
        batch = list(islice(self.iterator, self.size))
        if not batch:
            raise StopIteration
        return batch
def batched_students(students: Iterable[Student], size: int) -> Iterator[list[Student]]:
    yield from BatchIterator(students, size)
def infinite_ids() -> Iterator[int]:
    yield from count(1)
```

BatchIterator зберігає ітератор джерела та розмір пакета. __next__ отримує не більше size записів через islice. Останній пакет може бути неповним. Порожній пакет означає завершення. Значення size ≤ 0 відхиляється. batched_students делегує обхід за допомогою yield from.

Функція infinite_ids створює нескінченний генератор ID через count. У main він обмежується islice до п'яти значень. Виклик list(count()) без обмеження не завершиться та буде витрачати пам'ять.

4.2. Потокове читання

```
from collections.abc import Iterator
from pathlib import Path
def read_lines(path: Path) -> Iterator[str]:
    with path.open(encoding="utf-8-sig", newline="") as file:
        yield from file
```

Файл відкривається під час першого звернення до генератора. yield from передає рядки по одному, не викликаючи read() для всього файла. Кодування utf-8-sig підтримує UTF-8 із BOM і без нього. Параметр newline="" потрібний для коректної роботи csv.

4.3. Parsing, очищення та validation

```
import csv
from collections.abc import Iterable, Iterator
from dataclasses import dataclass
```

```
from math import isfinite
from typing import NamedTuple
FIELDS = ("student_id", "first_name", "last_name", "group", "average_grade")
class Student(NamedTuple):
    student_id: int
    first_name: str
    last_name: str
    group: str
    average_grade: float
@dataclass
class RecordCounts:
    total: int = 0
    valid: int = 0
    invalid: int = 0
def parse_rows(lines: Iterable[str]) -> Iterator[dict]:
    reader = csv.DictReader(lines)
    if tuple(reader.fieldnames or ()) != FIELDS:
        raise ValueError("Неправильний заголовок CSV.")
    yield from reader
def clean_rows(rows: Iterable[dict]) -> Iterator[dict]:
    for row in rows:
        yield {key: value.strip() if isinstance(value, str) else value
               for key, value in row.items()}
def validate_students(rows: Iterable[dict],
                      counts: RecordCounts | None = None) -> Iterator[Student]:
    if counts is None:
        counts = RecordCounts()
    for row in rows:
        counts.total += 1
        try:
            if None in row:
                raise ValueError("Зайві поля")
            student = Student(int(row["student_id"]), row["first_name"],
                              row["last_name"], row["group"],
                              float(row["average_grade"]))
            if (student.student_id <= 0 or not student.first_name
                    or not student.last_name or not student.group
                    or not isfinite(student.average_grade)
                    or not 0 <= student.average_grade <= 100):
                raise ValueError("Некоректні дані")
        except (ValueError, TypeError, KeyError):
            counts.invalid += 1
            continue
        counts.valid += 1
        yield student
```

csv.DictReader читає поля відповідно до заголовка. clean_rows прибирає крайні пробіли у значеннях. validate_students перетворює ID та бал на числа, відхиляє порожні й некоректні поля та підраховує записи. Student є незмінним NamedTuple. Некоректний заголовок спричиняє помилку, а некоректний запис пропускається.

4.4. Фільтрація, пошук і перетворення

```
from collections.abc import Iterable, Iterator
from stream_processor.parsers import Student
def filter_students(students: Iterable[Student], minimum: float = 0,
                    group: str | None = None) -> Iterator[Student]:
    for student in students:
        if student.average_grade >= minimum and (group is None or student.group == group):
            yield student
def normalize_names(students: Iterable[Student]) -> Iterator[Student]:
    for student in students:
        yield student._replace(first_name=student.first_name.title(),
                               last_name=student.last_name.title())
def search_students(students: Iterable[Student], student_id: int | None = None,
                    last_name: str | None = None) -> Iterator[Student]:
```

```
    for student in students:
        if student_id is not None and student.student_id != student_id:
            continue
        if last_name is not None and student.last_name.casefold() != last_name.casefold():
            continue
        yield student
```

Фільтрація використовує умову average_grade ≥ minimum. Якщо група не задана, дозволені всі групи. normalize_names повертає новий запис із нормалізованим регістром імені та прізвища. Пошук за прізвищем не залежить від регістру. Генератор пошуку читає джерело лише тоді, коли споживач просить наступний збіг.

4.5. Побудова lazy pipeline

```
from contextlib import closing
from pathlib import Path
from stream_processor.readers import read_lines
from stream_processor.parsers import parse_rows, clean_rows, validate_students, RecordCounts
from stream_processor.filters import filter_students, normalize_names
def build_pipeline(path: Path, minimum: float = 0, group: str | None = None,
                   counts: RecordCounts | None = None):
    with closing(read_lines(path)) as lines:
        rows = parse_rows(lines)
        cleaned = clean_rows(rows)
        valid = validate_students(cleaned, counts)
        selected = filter_students(valid, minimum, group)
        yield from normalize_names(selected)
```

closing закриває генератор читання, зокрема при достроковому завершенні зовнішнього конвеєра. У main короткі пошуки й вибірки також виконуються всередині with closing(...). Тому відкритий файл не залишається залежним від часу збирання сміття.

4.6. Схема потокової обробки

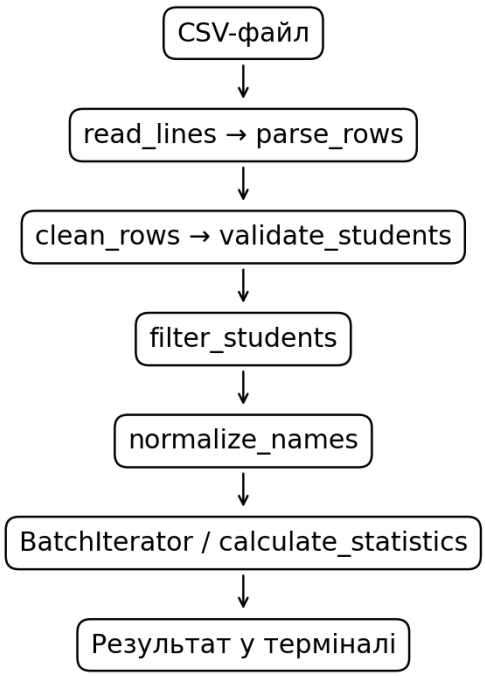


Рис. 1. Схема потокового конвеєра обробки даних студентів

Кожен етап передає запис наступному за потреби. Повний список проміжних результатів у lazy-варіанті не створюється. Агрегація зберігає лічильники й поточні значення, а пакетна обробка — лише поточний пакет.

4.7. Агрегація та найкращі N студентів

```
from collections import Counter
from collections.abc import Iterable
from heapq import nlargest
from stream_processor.parsers import Student
def calculate_statistics(students: Iterable[Student]) -> dict:
    count = 0
    total = 0.0
    minimum = maximum = None
    groups = Counter()
    for student in students:
        grade = student.average_grade
        count += 1
        total += grade
        groups[student.group] += 1
        minimum = grade if minimum is None else min(minimum, grade)
        maximum = grade if maximum is None else max(maximum, grade)
    return {"count": count, "average": total / count if count else None,
            "minimum": minimum, "maximum": maximum, "groups": dict(groups)}
def top_students(students: Iterable[Student], limit: int) -> list[Student]:
    if limit < 0:
        raise ValueError("Кількість студентів не може бути від'ємною.")
    return nlargest(limit, students, key=lambda student: student.average_grade)
```

Середній бал, мінімум і максимум визначаються за один прохід. Для порожнього потоку ці результати дорівнюють None. Counter зберігає лише кількість записів кожної групи. Пам'ять агрегації становить O(g), де g — кількість груп.

heapq.nlargest зберігає до N найкращих записів, тому весь набір не сортується. Для N, значно меншого за кількість записів m, пам'ять становить O(N), а час — O(m log N). Для визначення найкращих записів у невпорядкованому файлі необхідно прочитати весь потік. islice повертає перші записи за порядком файла і сам по собі не знаходить найбільші бали.

4.8. Generator expression та itertools

```
sizes = (len(batch) for batch in batched_students(build_pipeline(path), 1000))
grades = (student.average_grade for student in islice(stream, 5))
for previous, current in pairwise(grades):
    print(current - previous)
list(islice(infinite_ids(), 5))
```

Використано три засоби itertools. islice обмежує вибірку та розмір пакета; count створює послідовні ID; pairwise утворює пари сусідніх балів. Генераторні вирази передають розміри пакетів і бали без окремих списків.

4.9. Eager-реалізація

```
def read_eager(path: Path, minimum: float = 0, group: str | None = None,
               counts: RecordCounts | None = None):
    with closing(read_lines(path)) as source:
        lines = list(source)
    rows = list(parse_rows(lines))
    cleaned = list(clean_rows(rows))
    valid = list(validate_students(cleaned, counts))
    selected = list(filter_students(valid, minimum, group))
    return list(normalize_names(selected))
```

Eager-варіант виконує ті самі етапи, але матеріалізує їх результати у списки. Це дозволяє повторний обхід та індексацію, проте потребує пам'яті O(m). Lazy-варіант має одноразовий обхід і потребує нового конвеєра для повторної статистики.

4.10. Вимірювання часу та пам'яті

```
def measure(function, *args):
    gc.collect()
    start = perf_counter()
    result = function(*args)
    elapsed = perf_counter() - start
    gc.collect()
    tracemalloc.start()
    try:
        checked = function(*args)
        peak = tracemalloc.get_traced_memory()[1] / (1024 * 1024)
    finally:
        tracemalloc.stop()
    assert checked == result
    return result, elapsed, peak
```

Повний час вимірюється perf_counter без увімкненого tracemalloc. Пікова пам'ять визначається окремим повторним запуском. В обох випадках виконується повна агрегація, тому вимірюється споживання потоку, а не лише створення генератора. Порівнюються однакові дані з умовою бала не нижче 85; результати перевіряються на рівність.

tracemalloc вимірює виділення пам'яті Python, а не всю оперативну пам'ять процесу. MiB дорівнює 1 048 576 байтів. Створення CSV не входить до вимірювання. Час до першого результату вимірюється окремо. На час впливають навантаження комп'ютера та файловий кеш.

4.11. Достроковий пошук

```
def early_search(path, eager):
    counts = RecordCounts()
    if eager:
        students = read_eager(path, counts=counts)
        selected = list(search_students(students, student_id=2))
        result = next(iter(selected), None)
    else:
        with closing(build_pipeline(path, counts=counts)) as students:
            result = next(search_students(students, student_id=2), None)
    return result, counts.total
```

У файлі на 1 000 000 записів шукається ID 2. Lazy-пошук завершується після другого запису, тоді як eager-варіант спочатку створює повний список даних і список збігів. В обох випадках повертається той самий студент. Якщо збігу немає, lazy-пошук також повинен переглянути весь файл.

4.12. Тести

У test_pipeline.py є вісім тестів. Вони перевіряють валідацію й статистику, відбір та найкращі записи, лінивий пошук, пакетний ітератор, порожній файл і заголовок, CSV із лапками та некоректними полями, закриття файла після раннього виходу і рівність eager/lazy. Граничні бали 0 та 100 приймаються, nan та inf відхиляються.

5. Файл pyproject.toml

```
[build-system]
requires = ["setuptools>=70"]
```

```
build-backend = "setuptools.build_meta"
[project]
name = "student-stream-processor"
version = "0.1.0"
description = "Streaming student data processing"
readme = "README.md"
requires-python = ">=3.11"
authors = [{name = "Руссу Анастасія"}]
[project.optional-dependencies]
test = ["pytest>=8"]
[project.scripts]
student-stream = "stream_processor.main:main"
[tool.setuptools.packages.find]
where = ["src"]
[tool.pytest.ini_options]
testpaths = ["tests"]
```

6. Файл .gitignore

```
.venv/
__pycache__/
.pytest_cache/
*.egg-info/
build/
dist/
.vscode/
```

Середовище, кеші та результати збирання не відстежуються Git. CSV із вхідними даними включено до репозиторію, щоб програму можна було запустити одразу після встановлення.

7. Фрагмент README.md

README містить призначення програми, структуру каталогів, формат CSV та команди встановлення, запуску, експерименту й тестування.

```
# Потокова обробка даних системи обліку студентів
Лабораторна робота №3, варіант №1.
Авторка: Руссу Анастасія Сергіївна, ФЕП-32.
Викладач: Свелеба С. А.
Програма читає CSV поступово, перевіряє записи студентів,
виконує відбір, пошук, агрегацію та обробку пакетами.
```

8. Створення virtual environment та запуск

Команди для Windows виконуються у PowerShell із кореневого каталогу проєкту. Для ізоляції залежностей створюється .venv, після чого пакет установлюється в режимі редагування разом із pytest.

```
& "C:\Program Files\Python311\python.exe" -m venv .venv
.\.venv\Scripts\python.exe -m pip install -e ".[test]"
.\.venv\Scripts\python.exe -m stream_processor.main
.\.venv\Scripts\python.exe -m stream_processor.main --path data/sample.csv --batch-size 4
.\.venv\Scripts\python.exe -m stream_processor.benchmark
.\.venv\Scripts\python.exe -m pytest -q
```

Окрема активація не потрібна, оскільки використовується шлях до інтерпретатора середовища. Параметр -e пов’язує пакет із вихідним кодом, а [test] додає залежність для тестування. Параметри --group, --minimum, --limit, --student-id, --last-name та --batch-size дозволяють змінювати умови запуску.

9. Результат роботи програми

9.1. Малий набір даних

У sample.csv коректними є 6 із 10 записів. Сума балів становить 508, середній бал — 84,6667, мінімальний — 72, максимальний — 95. Кожна з трьох груп містить двох студентів. За умовою бала не нижче 85 відбираються записи з ID 1, 2, 3 і 5.

```
ПЕРЕВІРКА ВХІДНИХ ДАНИХ
Усього записів: 10; коректних: 6; некоректних: 4
Середній бал: 84.6667
Мінімальний бал: 72.0000
Максимальний бал: 95.0000
Кількість за групами: {'ФЕП-32': 2, 'ФЕП-31': 2, 'ФЕП-33': 2}

ПЕРШІ 5 ЗАПИСІВ ЗА УМОВОЮ: бал >= 85; група усі
ID       Прізвище та ім'я          Група          Бал
-----------------------------------------------------
1        Дорош Марія               ФЕП-32       86.50
2        Левицький Данило          ФЕП-31       95.00
3        Климчук Соломія           ФЕП-32       91.50
5        Романюк Олена             ФЕП-33       85.00
Відібрано: 4
```

Рис. 2. Перевірка записів, статистика та відбір студентів

Порядок найкращих студентів — ID 2, 3, 1, 5 із балами 95; 91,5; 86,5; 85. На відміну від попередньої роботи, тут використовується мінімальний допустимий бал, тому студент із балом, рівним порогу, входить до вибірки.

```
НАЙКРАЩІ 5 СТУДЕНТІВ ЗА ЦІЄЮ УМОВОЮ
ID       Прізвище та ім'я          Група          Бал
-----------------------------------------------------
2        Левицький Данило          ФЕП-31       95.00
3        Климчук Соломія           ФЕП-32       91.50
1        Дорош Марія               ФЕП-32       86.50
5        Романюк Олена             ФЕП-33       85.00

ПОШУК ID 2
ID       Прізвище та ім'я          Група          Бал
-----------------------------------------------------
2        Левицький Данило          ФЕП-31       95.00
Прочитано записів до результату: 2

ПОШУК ПРІЗВИЩА Дорош, до 5 результатів
ID       Прізвище та ім'я          Група          Бал
-----------------------------------------------------
1        Дорош Марія               ФЕП-32       86.50
6        Дорош Тарас               ФЕП-33       72.00

ПАКЕТИ: 2; записів: 6; останній пакет: 2
iter(iterator) is iterator: True
Розмір першого пакета через next(): 4
Решта записів: 2
Після вичерпання next(iterator, None): None
Перші 5 ID нескінченного потоку: [1, 2, 3, 4, 5]
Різниці сусідніх балів: [8.5, -3.5, -13.5, 7.0]
```

Рис. 3. Найкращі студенти, пошук та пакетна обробка

Пошук ID 2 читає лише два записи. Пошук прізвища «Дорош» повертає ID 1 та 6. За розміру пакета 4 утворюються пакети на 4 та 2 записи. Після завершення обходу next(iterator, None) повертає None.

9.2. Обробка великого файла

У students.csv міститься 100 000 коректних записів. Чотири групи мають по 25 000 студентів. За розміру пакета 1 000 утворюється 100 пакетів. Під час агрегації записи не накопичуються у повний список.

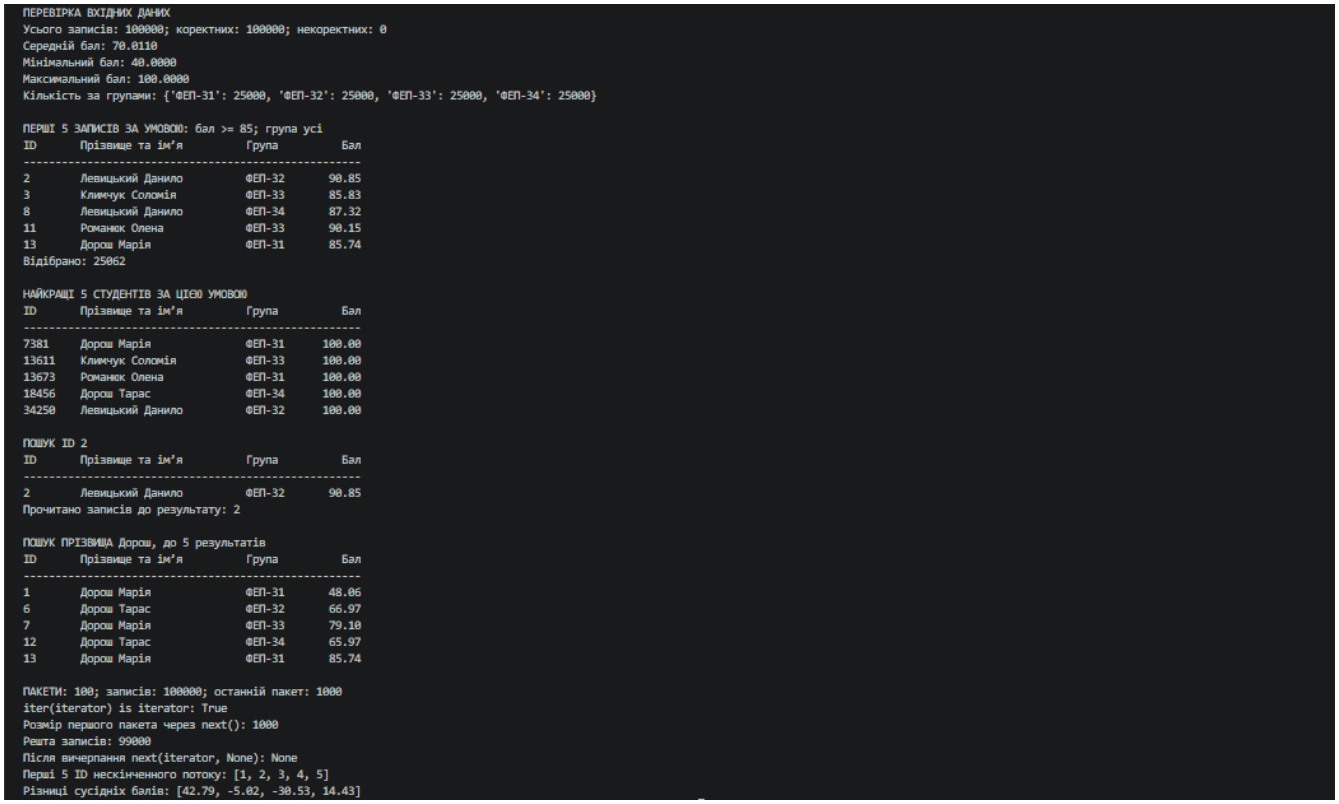


Рис. 4. Потокова обробка 100 000 записів

9.3. Порівняння eager та lazy

Для кожного розміру набору порівнюються повна агрегація, пікова пам'ять і час до першого відібраного запису.

Записів	Eager, с	Lazy, с	Перший eager, с	Перший lazy, с
10 000	0,0384	0,0351	1,8198	0,000283
100 000	0,3999	0,3804	0,392931	0,000383
500 000	2,5198	1,8198	2,265976	0,001011

Пікова пам'ять обох варіантів наведена окремо.

Записів	Eager, MiB	Lazy, MiB
10 000	10,7814	0,2276
100 000	106,5963	0,2274
500 000	534,0834	0,2272

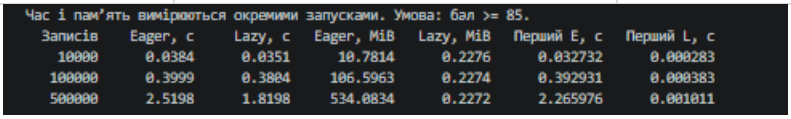


Рис. 5. Час, пікова пам'ять і час до першого результату eager та lazy

Lazy-обробка не зберігає проміжні списки, тому її пам'ять визначається поточним записом, буфером читання та кількістю груп. Eager-обробка зберігає набір і проміжні колекції. Lazy може швидше повернути перший результат, але не гарантує меншого повного часу. Остаточне порівняння часу виконується за виміряними значеннями.

9.4. Дострокове завершення

Спосіб	Записів у файлі	Прочитано	Час, с	Пік, MiB
Eager	1 000 000	1 000 000	7,655306	1266,6834
Lazy	1 000 000	2	0,000173	0,0433

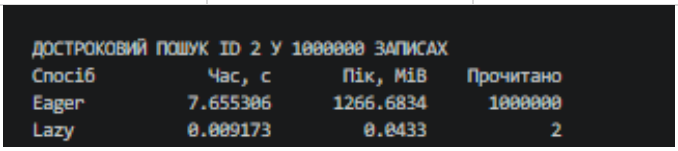


Рис. 6. Достроковий пошук ID 2 у файлі на 1 000 000 записів

Ранній вихід корисний, коли потрібен перший збіг поблизу початку файла. Для глобального максимуму, середнього значення та найкращих N студентів необхідно спожити весь потік. Після вичерпання генератора для повторного обходу створюється новий.

9.5. Перевірка коректності

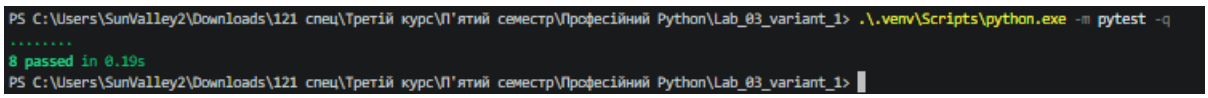


Рис. 7. Результат виконання автоматизованих тестів

Тести зіставляють результати з контрольними значеннями та перевіряють граничні випадки. Окремо перевірено, що після отримання першого збігу файл закривається, а кількість прочитаних записів не дорівнює розміру всього набору.

10. Робота з Git

Зміни зафіксовано трьома етапами — конфігурація проєкту; потокові модулі, дані й тести; запуск, експеримент і README. Звіт додається окремо після оформлення результатів запуску.

```
git add .gitignore pyproject.toml src/stream_processor/__init__.py
git commit -m "Create src layout and package configuration"
git add src data tests
git commit -m "Implement student streaming pipeline and validation tests"
git add src/stream_processor/main.py src/stream_processor/benchmark.py README.md
git commit -m "Add console output and eager lazy experiments"
```

10.1. Результат git status

```
On branch main
nothing to commit, working tree clean
```

10.2. Результат git log --oneline

```
f45215c Add console output and eager lazy experiments
01c29b4 Implement student streaming pipeline and validation tests
188b935 Create src layout and package configuration
```

11. Відповіді на контрольні запитання

1. Що таке iterable?

Об’єкт, для якого iter() повертає ітератор. Приклади — список, рядок і файл.

2. Що таке iterator?

Об’єкт, який зберігає стан обходу та повертає наступний елемент через next().

3. Чим iterable відрізняється від iterator?

Iterable є джерелом ітератора; iterator безпосередньо виконує послідовний обхід і може вичерпатися.

4. Для чого використовується iter()?

Для отримання ітератора з об’єкта, який підтримує ітерацію.

5. Для чого використовується next()?

Для отримання наступного значення. Можна передати значення за замовчуванням на випадок завершення.

6. Що робить __iter__()?

Повертає ітератор об’єкта. У власному BatchIterator повертає self.

7. Що робить __next__()?

Повертає наступне значення або піднімає StopIteration після завершення.

8. Що таке StopIteration?

Виняток, який повідомляє, що в ітераторі більше немає значень.

9. Як цикл for використовує iterator protocol?

Спочатку викликає iter(), потім послідовно next() і завершується при StopIteration.

10. Чому iterator часто називають one-shot object?

Його стан просувається вперед, а після завершення обхід автоматично не відновлюється.

11. Чому iter(iterator) is iterator?

Ітератор сам є об’єктом обходу, тому його __iter__ повертає self.

12. Чи можна повторно використати exhausted generator?

Ні. Для нового обходу потрібно знову викликати генераторну функцію.

13. Що таке generator function?

Функція, у тілі якої є yield або yield from і яка створює генератор.

14. Яку роль виконує yield?

Повертає чергове значення та призупиняє функцію зі збереженням стану.

15. Чим yield відрізняється від return?

yield призупиняє виконання, а return завершує функцію або генератор.

16. Коли фактично починає виконуватися generator function?

Під час першого next() або початку споживання циклом for.

17. Що зберігає generator між викликами next()?

Локальні змінні, стан виконання та позицію, з якої потрібно продовжити.

18. Що таке generator expression?

Вираз у круглих дужках, який створює генератор значень, наприклад (s.average_grade for s in students).

19. Чим generator expression відрізняється від list comprehension?

Генераторний вираз обчислює елементи за потреби, а list comprehension одразу створює список.

20. Що таке lazy evaluation?

Відкладене обчислення значень лише під час їх запиту споживачем.

21. Що таке eager evaluation?

Обчислення всіх результатів одразу, наприклад формування повного списку.

22. Які переваги lazy evaluation?

Менше проміжних даних у пам’яті, ранній результат, можливість дострокового завершення та роботи з нескінченними потоками.

23. Які недоліки lazy evaluation?

Одноразовий обхід, відкладені помилки, відсутність довільної індексації та потреба контролювати відкриті ресурси.

24. Що таке streaming?

Послідовна обробка даних невеликими частинами без завантаження всього набору.

25. Чому streaming важливий для великих files?

Дозволяє працювати з файлами, що не вміщуються у доступну оперативну пам’ять.

26. Чому read() може бути проблемою для multi-GB file?

read() без обмеження створює в пам’яті вміст усього файла.

27. Чому file object є iterable?

Файл підтримує ітераційний протокол і повертає рядки послідовно.

28. Для чого використовується yield from?

Для делегування генерації іншому iterable; у роботі передає рядки файла та пакети.

29. Що таке itertools?

Модуль стандартної бібліотеки з інструментами для побудови й поєднання ітераторів.

30. Для чого itertools.count()?

Для створення нескінченної арифметичної послідовності.

31. Чому count() потрібно обмежувати?

Нескінченний потік не має останнього елемента, тому його повне споживання не завершиться.

32. Для чого islice()?

Для отримання обмеженого фрагмента ітератора без матеріалізації всього джерела.

33. Для чого chain()?

Для послідовного обходу кількох iterable як одного потоку.

34. Як працює takewhile()?

Повертає початкові елементи, доки умова істинна, і зупиняється на першому порушенні.

35. Як працює dropwhile()?

Пропускає початкові елементи, доки умова істинна, а потім повертає решту.

36. Для чого accumulate()?

Для накопичувальних результатів, наприклад проміжних сум.

37. Для чого pairwise()?

Для отримання сусідніх пар елементів, наприклад для різниці балів.

38. Як працює groupby()?

Об’єднує сусідні елементи з однаковим ключем у групи.

39. Чому перед groupby() часто потрібне sorting?

Щоб усі елементи одного ключа стали сусідніми. Без сортування той самий ключ може з’являтися в різних групах.

40. Що робить product()?

Утворює декартів добуток вхідних послідовностей.

41. Чим combinations() відрізняється від permutations()?

combinations не враховує порядок елементів, permutations враховує.

42. Чому combinatorial iterators можуть бути небезпечними навіть при lazy execution?

Кількість результатів може бути дуже великою, тому повний обхід потребує значного часу; деякі інструменти також зберігають вхідні дані.

43. Що таке batching?

Поділ потоку на пакети обмеженого розміру.

44. Які переваги batching?

Обмежує робочу пам’ять і дозволяє передавати дані до зовнішніх систем частинами.

45. Що таке lazy pipeline?

Послідовність етапів, які передають значення за запитом без повних проміжних колекцій.

46. Які stages може містити pipeline?

Читання, parsing, очищення, validation, filtering, transformation, batching і aggregation.

47. Що є source у streaming pipeline?

Початкове джерело даних, у цій роботі — CSV-файл.

48. Що є transformation?

Перетворення запису, наприклад нормалізація регістру імені.

49. Що є sink?

Кінцевий споживач потоку — обчислення, запис у файл або виведення результату.

50. Чи гарантує generator менший total execution time?

Ні. Генератор може економити пам’ять, але мати більші накладні витрати на передачу елементів.

51. Яка основна memory advantage generator?

Не потрібно одночасно зберігати всі елементи та проміжні списки.

52. Що таке time to first result?

Час від запуску обробки до отримання першого придатного результату.

53. Чому lazy pipeline добре підтримує early termination?

Споживач може припинити запит нових значень, після чого решта джерела не обробляється.

54. Чому exception у generator може виникати із затримкою?

Тіло генератора виконується під час споживання, тому саме тоді запускається код, який може спричинити помилку.

55. Чому list(generator) може бути проблемою?

Це повністю матеріалізує потік і може вичерпати пам’ять.

56. Що трапиться при list(count())?

Список безперервно зростатиме; операція не завершиться нормально і може вичерпати пам’ять.

57. Як generator може впливати на lifecycle file resource?

Призупинений генератор може утримувати відкритий файл. Потрібні повне споживання або явне close(), зокрема через contextlib.closing.

58. Як тестувати невеликий generator?

Порівняти list(generator) з очікуваним малим списком і перевірити порожній потік, помилки та завершення.

59. Як тестувати infinite generator?

Обмежити споживання через islice й перевірити лише скінченний префікс.

60. Коли краще використовувати list, а коли generator?

Список зручний для повторного обходу та індексації. Генератор — для одноразової послідовної обробки великих або нескінченних потоків.

12. Висновок

У ході виконання лабораторної роботи розроблено потоковий модуль обробки даних студентів за варіантом №1. Реалізовано власний ітератор пакетів, генератори з yield та yield from, генераторні вирази й засоби itertools. Конвеєр читає CSV, перевіряє та перетворює записи, виконує відбір, пошук і агрегацію. На контрольному наборі середній бал становить 84,6667, а пошук ID 2 потребує читання двох записів. Eager-варіант зберігає проміжні списки, тоді як lazy передає записи поступово. Вибір способу обробки залежить від обсягу даних, потреби повторного обходу та можливості дострокового завершення.